

Rapport sur le Projet Tower Défense

Sommaire

- Introduction
- Parties du cahier des charges traitées
 - Environnement du jeu
 - Règles générale
 - Version terminale
 - Interface graphique et Controlleur
- Problèmes connus
- Problèmes rencontré
 - Imagemcon
 - Delta time
 - Fullscreen et Résolution
 - Commande Pour Lancer le Jeu
- Conclusion

Introduction

Ce rapport vise à fournir un aperçu des aspects du cahier des charges qui ont été traités lors du développement du jeu de Tower Défense.

Nous discutons également des problèmes connus , des problèmes rencontrés et des pistes d'extensions envisagées pour améliorer le jeu.

Parties du Cahier des Charges traitées

1. Environnement de jeu

Nous avons utilisé une classe Menu pour pouvoir créer l'environnement de jeu.

La classe est un JPanel dans lequel on utilise 3 boutons qui nous permettent de se diriger dans Home, Play, Settings grâce à un cardLayout. Puis un bouton QUIT (il fait que quitter le jeu, il a pas de panel principale). Les Boutons Home, Play, Settings sont contrôlés par différentes classes tels que Home.java, Play.java et Settings.java.

- La classe Home.java affiche l'image de Home.
- La classe Play.java permet au joueur de choisir la map, la difficulté, le personnages et le mode de jeu. Le jeu peut être lancé par un bouton "Go".
- La classe Settings.java qui permet au joueur de choisir la résolution de la fenêtre avec un combobox ou bien une résolution full screen avec un checkbox et 2 boutons, un bouton pour accepter la résolution et un bouton pour le full screen .

2. Règles générales

Il y a plusieurs facteurs qui changent l'expérience du jeu.

Commençons par la difficulté, qui s'explique lui-même avec son nom.

- Il change la difficulté pour tuer les monstres :
- Il y a un multiplicateur de dégâts dans la fonction monsterHurt, qui change les dégâts final (après le calcul de résistance) de monstre. Par exemple si la difficulté est facile le monstre va recevoir 1,5 fois plus de dégâts final, par contre dans le cas difficile, il va recevoir demi de dégâts final.
- Il change aussi indirectement leur niveau, et donc les points de vie et de vitesse, car chaque vague augmente avec la limite du niveau soit 3.

Le temps :

- d'apparition entre chaque monstre,
- Entre les vagues,
- Entre les séries.

Tous ces attributs sont dans la classe MonsterSpawner qui est produit juste avant de lancer la partie du jeu. Selon la difficulté :

- le temps d'apparition pendant une série change de 0,5 seconde à 1,5 seconde .
- le temps entre les vagues, qui veut dire le temps entre les groupes des séries, change de 3 secondes à 7 secondes.
- le temps entre les séries ou bien les groupes de monstres qui attaquent dans un changement de 15 à 25 secondes.

Ayant une boucle des temps de série-série-vague, tuer 20 monstres(séries de 5 et 15) et faire une stratégie pour mettre/upgrade les tours dans 18 secondes et 32 secondes changent trop la difficulté du jeu. De plus, il y a quelque chose de plus grave : Le temps d'apparition des monstres. Il faut très bien faire le choix des circuits, sinon les monstres vont passer sans problème.

- Le nombre de vagues maximum.

Il est en relation avec le cas ci-dessus. En plus leurs attributs augmentent chaque vague, donc certainement après un certain nombre de vagues il sera trop difficile de les arrêter.

- Il y a plusieurs maps qui forcent à changer de stratégie.

- Il est choisi dans le menu play. Sa bouton change le texte d'un attribut JLabel() qui est utilisé après pour avoir des données des fichiers txt dans le ressources/maps, pour être changé en un tableau de tableau de cellules qui contient tout l'information du map.

- Presque tout est fait dans MapConfig, d'abord il prend nom de map et fait son path. Après avoir ouvert le fichier, il le lit et avant de faire des cellules il choisit la longueur et hauteur du map(tableau de tableau de cellules). Puis selon les nombres dans le fichier txt il produit les cellules et met dans le map.

Le mode de jeu peut changer énormément l'expérience du jeu si on veut plus ou moins de difficulté, ou bien si on veut voir où on peut aller jusqu'à.

- Le mode normal est où le nombre des vagues est limité. On peut gagner le jeu dans ce mode en survivant jusqu'à la vague limite. Ça change aussi la difficulté, car les niveaux des monstres dépendent du nombre de vagues. Le mode facile a 4 vagues, qui veut dire qu'on va voir des monstres de niveau maximum pendant 1-2 vagues, pendant que dans le mode hard où il y a 8 vagues, le demi de jeu passe avec des monstres de niveau max en plus des attributs amélioré à cause du nombre des vagues.

- Ayant un nombre de vagues infini, le mode marathon est le meilleur choix pour voir la limite pas que du jeu mais aussi notre. Le nombre de vagues maximum est assigné à -1 dans ce mode c'est-à-dire le nombre de vagues ne va jamais l'atteindre en commençant par 0.

Tous les personnages dans le jeu augmentent les dégâts des tours un peu. La choix de caractère est vital pour la stratégie qu'on va suivre.

- Les caractères Soldat et Archer augmentent le dégâts des tours de Soldat et Archer de façon très remarquable. Si on veut choisir un de ces caractères, on voudrait bien privilégier ses tours qui sont très moins chers à construire et upgrade. Ça peut être très utile notamment pendant les premières vagues, qui les rendent des choix excellents pour la difficulté facile.

- La caractère villageois peut être n'a pas trop de boost pour le dégâts des tours, mais il a sa moyenne très important pour faciliter les acheter et les upgrade: argent. Il découpe l'argent à donner en deux, qui facilite d'acheter beaucoup plus de tours au début de jeu. Il aide aussi à pour les upgrades des tours, qui sont très chers pour le canon et la catapulte.

- Le commandant est peut être le plus valable des autres car il a une spécialité très très importante: Il fait attaquer les tours deux fois plus rapidement. Ceci fait une différence énorme quand il y a beaucoup de catapultes et canons, qui sont des tours de vitesse d'attaque très lents par rapport aux autres.

3. La version terminale

Tout d'abord, nous avons fait la version terminale du jeu avec les classes primitives `Monster.java`, `Tour.java` et `Character.java` sauf les classes `MonsterPathFinding.java` et `MonsterSpawner.java` qui sont conservées sauf des petits changements. Avec ceci, nous avons ajouté quatre autres classes, `Jeu.java`, `Plateau.java`, `Player.java` et `TerminalAffichage.java` nous permettant de pouvoir interagir avec le joueur dans le terminal et respecter l'ancienne version du jeu.

Deuxièmement, nous allons expliquer l'utilité de chaque classe. Tout d'abord, la classe `Jeu.java` se compose de la partie logique du jeu. Ensuite, la classe `Plateau.java` comporte de l'affichage du jeu dans la console à l'instance. Puis, la classe `Player.java` se compose de l'interaction entre le joueur et la console. Enfin, la classe `TerminalAffichage.java` comprend le lancement de la version terminal.

4. Interface graphique et Controller

Après avoir cliquer sur "Go!", `GameView`, `JFrame` qui contient le menu et le jeu affiche produit une instance de `GameWholeScreen` et en utilisant les données de retour de bouton "Go!", il la laisse produire ses parties de "Modèle et Vue" (`Game` et `Gamescreen`). Enfin il fait afficher `GameWholeScreen` en utilisant `cardLayout` et lance le thread `game_thread` qui fait les updates de `GameWholeScreen`.

Dans cette instance, les objets de `Game` et `GameScreen` font leurs updates dans un fonction liée à `game_thread`. Il y a aussi des boutons de partie en bas, utilisés pour acheter/upgrade un tour, pause où quitter la partie. Il y a aussi un panel qui contient un message expliquant le statut de la partie: argent, vie de la base, nombre

de vague et des messages comme “Vous n’avez pas assez d'argent pour upgrade ce tour.”.

Dans Game, il y a la logique de jeu. Il gère les objets de Monster, Tour, Character, MonsterSpawner, MapConfig. Le mouvement des monstres, tire des tours, voir si la partie est perdue etc sont tous dans cette classe. Il relie tout et met dans une fonction d'update, qui est utilisée dans GameWholeScreen.

La partie graphique est faite dans la classe GameScreen. Il y a trois classes qui gèrent trois types d'objet dans le jeu: Caro-Cellule, MonsterGraphics-Monster, TourGraphics-Tour. Ces classes affichent ces objets dans l'écran en utilisant ses listes dans Game et MapConfig. GameScreen met les updates(affichages) de ces classes pour le mettre dans update de GameWholeScreen.

Problèmes connus

- Problème de message d'upgrade de tour

Problèmes rencontrés

Imagelcon

Nous avons rencontré un problème lors de la configuration des images des monstres. Nous avons voulu utiliser des fichiers .gif pour pouvoir représenter nos 4 monstres avec la classe Imagelcon. Nous avons voulu utiliser la classe Imagelcon car sur les différents forums que nous avons visités il nous recommandait d'adopter cette classe. Mais cela n'a pas voulu fonctionner, nous ne voyons pas les images des monstres lorsqu'on lance le jeu. Alors, nous avons essayé de comprendre pourquoi cela ne voulait pas fonctionner avec des débogages. Néanmoins, nous rencontrons toujours la même quand bien même que nous savions que nos monstres se déplacent dans notre carte. Après cela nous avons donc dû changer nos fichiers .gif en utilisant des fichiers .png pour pouvoir enfin faire afficher nos monstres avec la classe BufferedImage.

Delta time

Après avoir construit le thread du jeu, on a vu que `deltatime` ne marchait pas bien. D'abord on a vu que le temps coulait très vite, donc on a essayé de trouver comment le limiter. Après avoir cherché la solution pour 1-1.5 jours(1-1.5 business days), on a décidé de regarder des vidéos youtube et nous en avons trouvé une de la chaîne de RyiSnow. Après avoir regardé la vidéo on a décidé que utiliser `nanotime` à la place de `milliseconds` est mieux. Puis on a fait un limiteur de fps en essayant de trouver les bons nombres. Après la grande défaite, on a encore regardé la vidéo youtube et résolu le problème.

Fullscreen et Resolution

On peut vraiment dire qu'on a eu beaucoup des problèmes et des heures passées dans `stackoverflow` pour ces problèmes. Pendant qu'on faisait les paramètres, on a remarqué que quand on change la résolution, l'image dans home ne se met pas à jour. On a essayé de changer la grandeur d'image, réinitialiser l'image etc mais rien n'a marché. Enfin, on a fini par réinitialiser tout le menu. Même quand on a réinitialisé le menu, il ne marchait pas bien. Après avoir joué avec le `width` et `height` de `frame` et `main_panel`, enfin on a trouvé la solution de problème de résolution. Malheureusement c'était pas la fin des problèmes. On a eu encore des problèmes sur la résolution, mais cette fois c'était sur fullscreen. Et la source du problème était quoi? Image de menu. Encore. Après avoir passé des heures dans `StackOverflow`, on a trouvé une solution de deux lignes. On pensait que tout était résolu, jusqu'à ce que l'on lance le jeu. Le jeu prenait la résolution de l'ancienne fenêtre. On a essayé encore beaucoup de choses. Changer les grandeurs, mettre des attributs pour garder la grandeur finale, essayer de initialiser l'instance `GameWholeScreen` (où l'écran du jeu est produit) avant et après la commande `pack()`, qui était un fixé pour la résolution. Enfin, on a trouvé une page de `StackOverflow`, qui disait qu'il fallait fixer la taille avant de faire `pack()`. On a essayé cette solution, et voilà! La solution du problème causé par une solution de deux lignes était toujours deux lignes.

Commande Pour Lancer le Jeu

On a passé 1 jour pour trouver les bonnes commandes pour pouvoir lancer le jeu depuis un terminal. Nous avons essayé de faire un javac pour tous nos fichiers puis un java gui.App mais cela nous renvoyait une erreur soit disant que le fichier App n'existait pas dans le package gui. En plus, il n'arrivait pas à faire les compilations car il trouvait pas les autres classes. Alors nous avons commencé à chercher sur StackOverflow pour pouvoir trouver une réponse à notre problème. Comme toujours, après avoir bien utilisé les souris de nos ordinateurs, le héros de tous les développeurs nous a aidé. On a fini par trouver 3 commandes qui produisent le répertoire de compilation, compile les fichiers de java dans toute l'arborescence et qui lance le jeu.

Extension

- Ajouter des images pour les personnages ce qui permettra de mieux visualiser les personnages avec une option de skin.
- Ajouter différents thèmes pour différents événements. Par exemple, ajoutez un thème Noël dans lequel l'herbe, les arbres, l'eau et les tours pourraient recouvrir de neige ou bien un thème Halloween avec les arbres avec des oranges, des chauves-souris, etc.
- Embellir la pause avec un bouton reprendre et un bouton quitter la partie
- Embellir le game over avec un bouton de redémarrage et un bouton de menu de retour.
- Ajout de score et highscore.
- Ajouter le son.
- Enlever des tours
- Ajouter une sauvegarde

Conclusion

Pour conclure, nous avons bien aimé faire ce projet car cela nous a permis de nous familiariser avec java, même si à certain moment les bugs nous faisaient mal à la tête. De plus, nous nous sommes bien amusés à faire ce projet.